Who Wrote It? Identifying LLMs from Their Responses

Author: Kanon Tsuda

GitHub: [kft5329/cmpsc448-llm-classification: CMPSC 448 project: Identifying LLMs from their responses using CNN and RNN]

1. Problem Definition and Dataset

Task. Different LLMs may leave stylistic and structural fingerprints in their text. Given a response (and optionally the user prompt that produced it), we predict which LLM family wrote it. I study three research questions: **RQ1** can the generating LLM family be identified from the response alone; **RQ2** does the prompt help (input only vs. output only vs. input + output); and **RQ4 (extra credit)** which characteristics distinguish the LLMs, and what happens when those signals are removed.

Data source. I use the public `lmsys/chatbot_arena_conversations` dataset (Hugging Face, 33k rows, gated). Each row contains one user prompt answered by two different anonymous LLMs, together with the model names. The dataset is not redistributed in my repository; it can be rebuilt with the provided scripts after accepting the dataset terms.

Construction.

- Kept English, single-turn conversations only, and removed conversations flagged by the dataset's moderation field, to avoid sensitive content. Each row yields two samples (model A and model B) of the form (LLM_name, LLM_input, LLM_output): 51,822 samples from 25,928 unique prompts, covering 20 models.
- **Classes are LLM families** (5 classes): GPT (gpt-4, gpt-3.5-turbo), Claude (claude-v1, claude-instant-v1), PaLM (palm-2), Llama-derived (vicuna-13b, koala-13b, alpaca-13b), and Pythia-based (oasst-pythia-12b, dolly-v2-12b). Grouping by family follows the assignment's "at least 3 LLM families" requirement. The Llama-derived and Pythia-based assignments reflect my understanding of the models' base architectures.
- **Balancing:** each family has exactly 2,298 samples (11,490 total), drawn in equal numbers from its member models (PaLM, with 2,324 samples available, is the limiting class). Chance accuracy is 20%.

- **Split:** 80/10/10 into train (9,170), validation (1,158) and test (1,162). The split is done **by prompt ID**: because the same prompt is answered by two models in this dataset, a random sample-level split would leak prompts across splits.

Limitations of the data. The models are from 2023, the data is English and single-turn only, and family classes mix several related models.

2. Models and Training

Preprocessing. Regex tokenization into words, punctuation marks and newlines (a special `<nl>` token); case is preserved, since capitalization and formatting may carry stylistic signal. The vocabulary is built from the training set only (maximum 30,000 tokens, minimum frequency 2). Inputs are truncated to 256 tokens for the output and 128 tokens for the prompt; in the input + output setting they are concatenated with a `<sep>` token.

CNN (TextCNN). Embedding (128-d, randomly initialized) → three parallel Conv1D layers with kernel sizes 3, 4, 5 (128 filters each, ReLU) → max-over-time pooling → dropout 0.5 → linear layer (5 classes).

RNN (BiLSTM). Embedding (128-d) with dropout → one bidirectional LSTM layer (hidden size 128) → concatenation of the final forward and backward states → dropout 0.5 → linear layer (5 classes).

Training details (both models). Adam, learning rate 1e-3, batch size 64, cross-entropy loss, gradient clipping at 5.0, at most 20 epochs, early stopping when validation macro-F1 does not improve for 3 epochs, and the best-validation checkpoint is evaluated once on the test set. No hyperparameter search was performed (single configuration), and training was done on CPU. The CNN was trained with 3 random seeds; the BiLSTM with 1 seed because of compute time.

Baseline. TF-IDF (word uni- and bi-grams) + logistic regression, for reference.

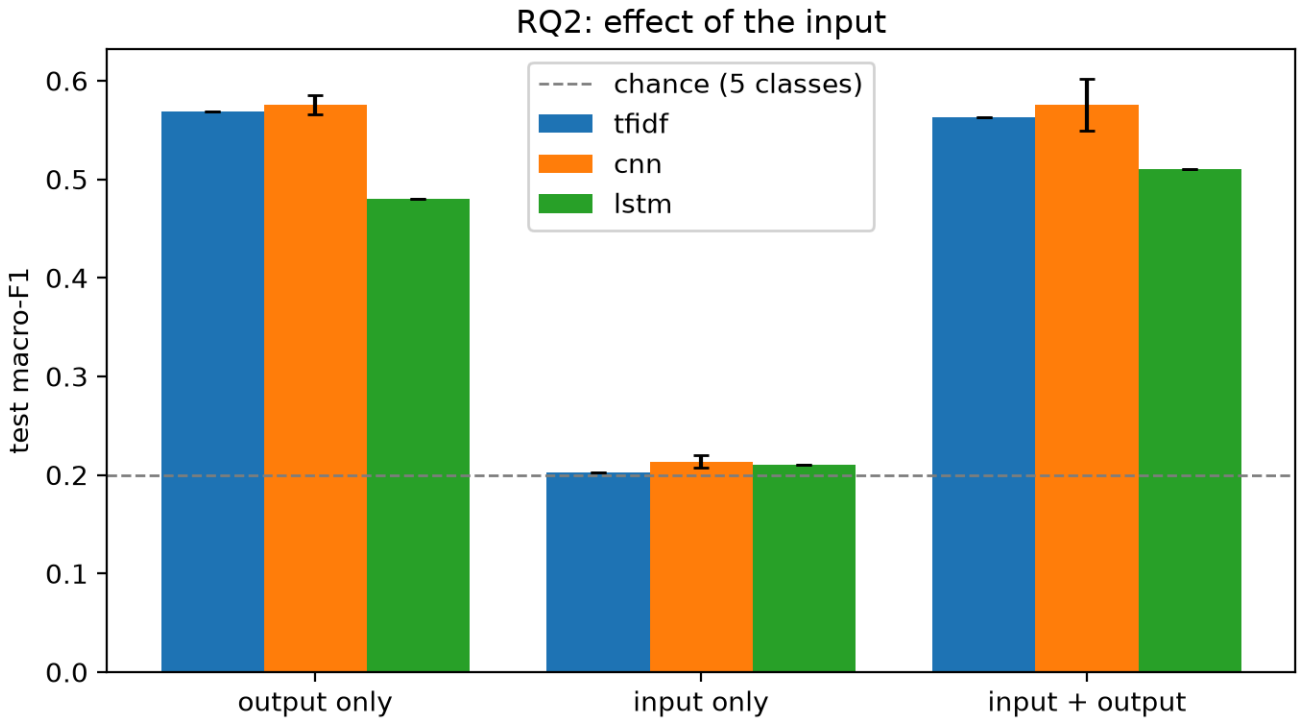
3. Results (RQ1 and RQ2)

All numbers are on the test set (1,162 samples). Macro-F1 (%), chance = 20%. CNN: mean ± std over 3 seeds; BiLSTM and TF-IDF: single run.

Model	Output only (RQ1)	Input only	Input + Output
TF-IDF + LR	56.9	20.3	56.3

Model	Output only (RQ1)	Input only	Input + Output
CNN	57.6 ± 1.0	21.4 ± 0.6	57.6 ± 2.6
BiLSTM	48.0	21.0	51.0

Accuracy (%) follows the same pattern: CNN 57.5 ± 1.1 / 21.7 ± 0.8 / 57.7 ± 2.0; BiLSTM 49.7 / 21.3 / 51.7; TF-IDF 56.7 / 20.3 / 56.5.

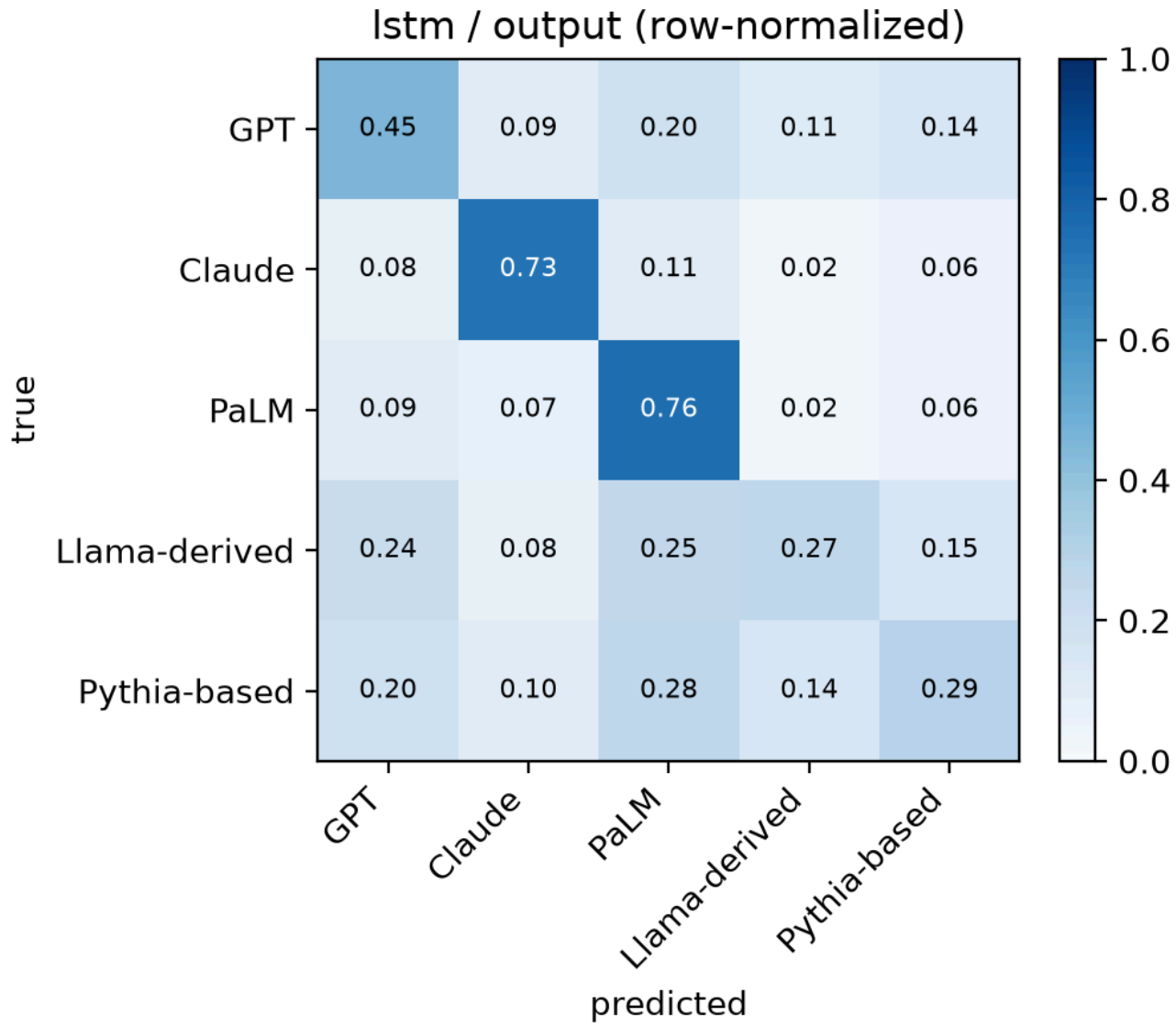


RQ1: the LLM family can be identified from the response. All models are far above the 20% chance level (CNN 57.6%). However, the CNN is not better than the simple TF-IDF baseline (57.6 vs. 56.9), so on this relatively small dataset (9k training samples) deep models bring no clear advantage; the CNN also overfits (training loss falls to about 0.3 while validation performance plateaus). The BiLSTM is lower (48.0); it was stopped early, run with a single seed and not tuned, so I do not claim that RNNs are inherently worse here. With 1,162 test samples the standard error of an accuracy near 57% is about 1.5 points, so differences below roughly 3 points should not be over-interpreted.

Per-family F1 (output only). Claude and PaLM are the easiest to identify; GPT is intermediate; the Llama-derived and Pythia-based classes are hardest.

Family	TF-IDF	CNN	BiLSTM
GPT	0.527	0.455	0.434
Claude	0.795	0.817	0.706
PaLM	0.682	0.715	0.579
Llama-derived	0.423	0.428	0.351
Pythia-based	0.418	0.464	0.333

(CNN and BiLSTM: seed 0.)



RQ2: the prompt does not help. (i) Predicting the family from the prompt alone is essentially at chance for all three models (20.3–21.4%). (ii) Adding the prompt to the response does not give a consistent improvement: CNN 57.6 vs. 57.6, TF-IDF 56.3 vs. 56.9 (slightly lower), BiLSTM 51.0 vs. 48.0 (single run, within noise). (iii) The near-chance input-only result is expected for this dataset: every prompt is sent to two different models, so the prompt carries almost no information about which model answers. The input-only CNN's training loss falls to about 0.85 while validation accuracy stays near 22%, i.e. it memorizes training prompts without learning anything that generalizes.

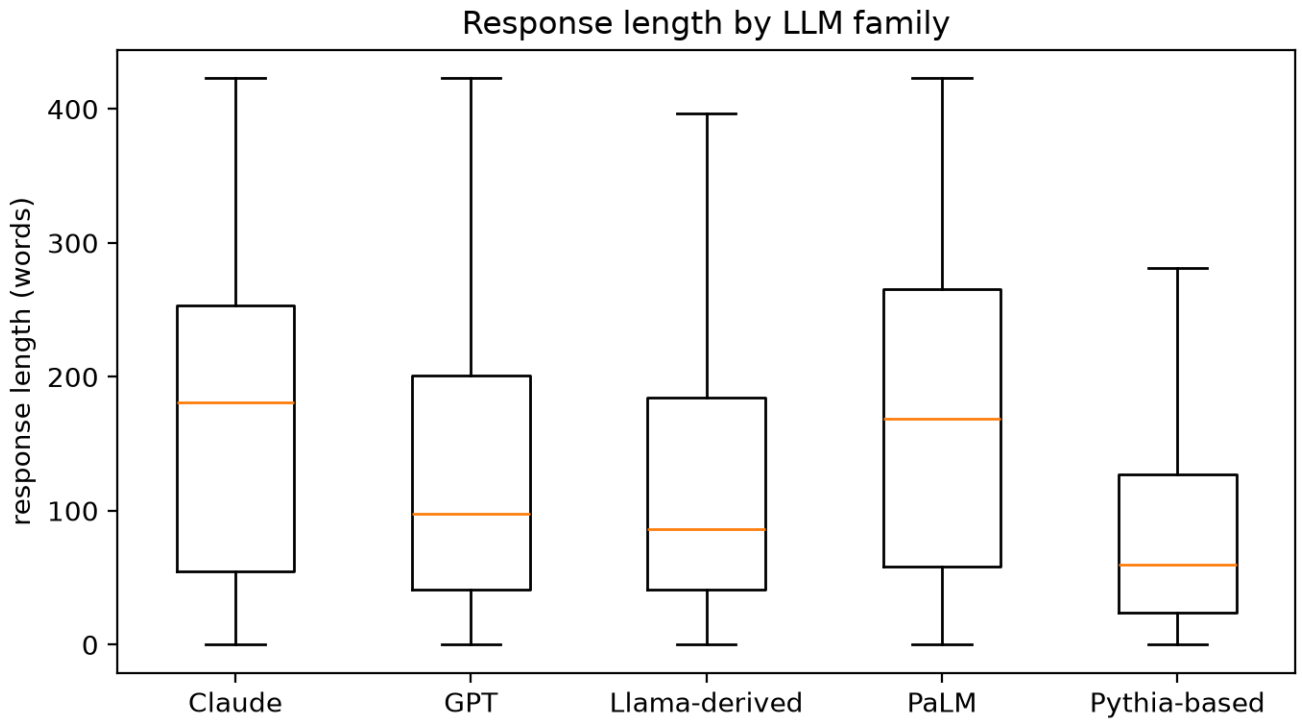
4. In-depth Analysis (RQ4)

I ask which characteristics distinguish the families, using outputs only. Details and code are in `04_rq4_analysis.py`.

(A) Descriptive statistics. Mean feature values per family (all 11,490 samples):

Family	Words	Words/sentence	Type-token ratio (first 100 words)	Lines	Bullets	Bold (**)	Apology	"As an AI"-type
Claude	164.5	22.2	0.684	10.0	2.40	0.001	6.4%	1.3%
GPT	136.6	21.4	0.715	8.0	0.50	0.017	3.5%	7.3%
Llama-derived	117.5	23.3	0.682	5.6	0.32	0.008	3.8%	4.0%
PaLM	173.2	21.5	0.647	9.3	2.01	0.755	1.4%	7.7%
Pythia-based	92.6	21.5	0.712	4.8	0.35	0.009	2.2%	1.7%

Claude and PaLM write long, list-heavy answers; PaLM is the only family that uses bold markup regularly; Claude apologizes most often; Pythia-based models give the shortest answers.



(B) Handcrafted features only. A random forest on 25 handcrafted features (length, lexical, structure/formatting, punctuation and phrase features; no raw text) reaches **54.6% macro-F1**, close to the neural models, so surface statistics alone carry much of the signal. Feature-group ablation (test macro-F1, %):

Setting	Drop the group	Use only the group
(all features)	54.6	-
Length	52.2 (−2.4)	31.6
Lexical	51.8 (−2.8)	30.2
Structure / formatting	44.3 (−10.3)	40.2
Punctuation	49.9 (−4.7)	33.7
Phrases	53.3 (−1.3)	17.6

Structure/formatting is both the strongest group on its own and the one whose removal hurts most. Phrase features alone are at chance. Removing length or lexical features changes little, but this should not be read as "length is unimportant": other groups (e.g.,

number of lines) partly encode length. Random-forest feature importances are spread fairly evenly and are known to favor continuous features, so I rely on the group ablation rather than on the importance ranking.

(C) Removing signals from the text. To test which signals the text classifier uses, I train TF-IDF + logistic regression (word uni-/bi-grams, with punctuation and newline tokens kept) on transformed texts. The reference ("original") reaches 61.6 macro-F1. This is higher than the default TF-IDF baseline in Section 3 (56.9), which ignores punctuation, newlines and case; the two setups differ in these respects, so I do not attribute the gap to a single cause.

Transformation	Macro-F1	Change vs. original
Original	61.6	-
Remove markdown symbols	59.6	−2.0
Remove newlines/extra whitespace	59.6	−2.1
Remove m arkdown and newlines	57.6	−4.0
Lowercase and remove punctuation	57.5	−4.1
Keep only the first 50 words	53.1	−8.5
Drop the first sentence	59.2	−2.5

Formatting and punctuation are useful signals but not the whole story: accuracy stays near 58% even after both are removed, so word choice itself is informative. The largest drop comes from truncating to 50 words, but this removes both length information and the amount of text available, so it cannot be attributed to length alone. Dropping the first sentence (a typical canned opener) costs little.

Conclusion: the signals are both structural (length, lists, formatting, punctuation) and linguistic (vocabulary), with structure the stronger of the handcrafted signals.

5. Lessons and Experience

- **Data design matters as much as the model.** Because the dataset pairs two models per prompt, the prompt is uninformative, which explains the RQ2 result.

- **Simple baselines are strong.** TF-IDF + logistic regression matched the CNN. With about 9k training samples, the neural models overfit quickly, and I did not tune them, so my conclusions apply to this configuration only.
- **Uncertainty matters.** With a test set of about 1,160 samples, differences of a few points are within noise; running multiple seeds (CNN) and reporting mean ± std made the conclusions more reliable. The BiLSTM and the baselines were run once, which is a limitation.
- **Practical issues.** Accessing the gated dataset required authentication, and CPU-only BiLSTM training took a lot of time since it was so slow and limited the number of experiments.
- **Future work.** I would like to integrate more seeds for the BiLSTM, and work on RQ3 too.
- **Use of AI assistants.** I used Claude to help plan experiments, write and debug code. I reviewed, ran and verified the code and results reported here.